Mini Plants vs Zombies Game

Grade 9 Computer Science 3 | First Quarter | Pair Activity | Alternative Assessment | 100 points

Create a short Python game that runs in the terminal. One Plant protects a lane from one Zombie. Use classes and objects to make them interact. You will plan, draw a UML class diagram, code, test, and explain your work with your partner.

Partners: Segotier & Vitudio
Section: 9 Beryllium Date: 9/30/26

Your game

- Use one lane, two Plant objects, and one Zombie object. Give the plants different damage values. Graphics, menus, and keyboard input are not required.
- Each living plant attacks once per turn, in order. If the Zombie reaches 0 health, stop that turn. Otherwise, it moves one step closer when its distance is greater than 0. At distance 0, it attacks the first living plant instead.
- If the first plant is defeated, the Zombie targets the second plant on later turns. A defeated plant cannot attack. Show each action, all health values, and the Zombie distance.
- The plants win when the Zombie reaches 0 health or less. The Zombie wins when both plants reach 0 health or less. Stop immediately when either side wins and print the result.
- Choose starting values that let the Zombie reach and attack a plant in at least one test. You may change the values for other test cases.

Minimum class requirements

Class	Attributes	Methods
Plant	name, health, damage	attack(zombie), take_damage(amount)
Zombie	name, health, damage, distance	move(), attack(plant), take_damage(amount)

Use __init__ and self to store each object's attributes. Create both plants from the same Plant class, for example, Peashooter with 15 damage and Snow Pea with 10 damage. Different damage values are enough; a special skill is optional. An attack must pass the other object to a method, such as plant.attack(zombie) and zombie.attack(plant). The target object's take_damage method updates its health.

A Game class is optional. A turn loop in the main program is enough. Do not place every action inside one long block of code while leaving your class methods unused.

Before coding

Complete these four short planning responses. A few clear words or numbered steps are enough.

Thinking skill	What to write
Decomposition	List the smaller parts needed to make the game work.
Pattern recognition	Name actions or checks that happen again each turn.
Abstraction	List the information each character actually needs.
Algorithm design	Write the game steps in order, including when it stops.

UML diagram

Draw a UML class diagram with Plant and Zombie, their required attributes and methods, and a line between the classes labeled “interacts with.” Use any online tool to draw your diagram. One Plant class represents both plant objects; you do not need a separate class for each plant. Multiplicity is optional. Your diagram should match your code.

Work plan

Session	Time	What to do
1	10 minutes	Plan the game using the four thinking skills.
1	10 minutes	Draw the UML diagram and show the association.
1	25 minutes	Code the two classes, create three objects, and try one plant attacking the Zombie.
1	5 minutes	Show the teacher both plants, the Zombie, and one working interaction.
2	25 minutes	Finish the turn loop, two plant attacks, movement, target selection, and winner.
2	10 minutes	Run and record the test cases below; correct any errors.
2	15 minutes	Demonstrate the game. Both partners explain part of the code.

Testing checklist

Run the program with values that show each situation. Write the values you changed and the result beside each item. You may temporarily change starting values to test a case.

☐ The two plants have different damage values: _____

☐ Plants defeat the Zombie: _____

☐ Zombie reaches distance 0 and damages a plant: _____

☐ Zombie targets the second plant after the first is defeated: _____

☐ Game stops when the Zombie or both plants are defeated: _____

What to submit and show

One markdown planning sheet with the four computational thinking responses.

One UML diagram with the two classes and their relationship.

One Python file that runs in the terminal, plus the completed testing checklist.

A short live demonstration. Each partner should explain at least one class or method, why the two plants can have different damage values, and how the objects interact.

Each student also submits an individual reflection of 5 to 7 sentences after the demonstration. In your own words, describe your contribution, one problem you encountered and how you addressed it, and what you learned about classes, objects, or how the Plant and Zombie interact. Write your name on your reflection. You may finish it outside the two coding sessions.

Before submitting, check that your program runs without errors, the output is readable, and the UML names match your code. Write both names on the planning sheet and in a comment at the top of the Python file.

How and where to submit

Name your folder **aa_partnerlastname_yourlastname**

Place all your files inside the folder (python file, markdown planning sheet, UML diagram)

Upload the folder to your respective GitHub accounts

Submit your individual reflection in GForm (Until Friday, 11:59 pm)

Scoring rubric

Criterion	How points are earned	Points
Computational thinking	Decomposition 3; patterns 2; abstraction 2; algorithm 3	10
UML diagram	Plant 5; Zombie 5; attributes, methods, and association 5	15
Classes and objects	Plant 5; Zombie 5; two different Plant objects and Zombie 5; use of __init__ and self 5	20
Game behavior	Both plants attack 5; move 5; target and attack 5; health 5; winner and stop 5	25
Object interaction	Plant and Zombie methods receive and affect the other object	10
Code readability	Clear names, indentation, and sensible organization	5
Testing	Required cases demonstrated and recorded	5
Pair demonstration	Both partners explain their work accurately	5
Individual reflection (Due Friday)	Own contribution 2; challenge and response 2; learning 1	5
Total		100

Optional challenge: After the required game works, add one extra skill to a plant, such as a stronger attack every third turn, or add a simple text lane. Explain the skill and show that the basic game still works. Up to 5 bonus points may be awarded, with the final score capped at 100. Extra features do not replace required features.